Day 1 :

```
StringBuilder word = new StringBuilder();
      for(int i = 0; i < S.length(); i++) {
      char ch = S.charAt(i);
      if(Character.isUpperCase(ch) && i != 0) {
         System.out.println(word.toString());
         word = new StringBuilder();
      }
word.append(ch);
   }
   System.out.println(word.toString());
```

Day 2 :

```
Integer[] arr = new Integer[N];
   for (int i = 0; i < N; i++) {
      arr[i] = A[i];
   }

   Arrays.sort(arr, (a, b) -> {
      int d1 = String.valueOf(a).length();
      int d2 = String.valueOf(b).length();
      return d1 - d2;
   });

   for (int i = 0; i < N; i++) {
      A[i] = arr[i];
   }

   return A;
```

Day 3 :

```
import java.util.Scanner;

class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int subtract(int a, int b) {
        return a - b;
    }

    int multiply(int a, int b) {
        return a * b;
    }

    int divide(int a, int b) {
        if (b == 0) {
            System.out.println("Err: division by zero");
            return 0;
        }
        return a / b;
    }
}

public class Main {

    public static int calculate(int a, int b, char op) {
        Calculator calc = new Calculator();
```

```
    switch (op) {
        case '+': return calc.add(a, b);
        case '-': return calc.subtract(a, b);
        case '*': return calc.multiply(a, b);
        case '/': return calc.divide(a, b);
        default: return 0;
    }
}

// Non editable starts here
public static void main(String[] args) {

    try (Scanner sc = new Scanner(System.in)) {

        if (!sc.hasNextInt()) return;
        int a = sc.nextInt();

        if (!sc.hasNextInt()) return;
        int b = sc.nextInt();

        sc.nextLine();

        String opLine = sc.nextLine();
        if (opLine == null || opLine.length() == 0) return;

        char op = opLine.charAt(0);

        if (op == '/' && b == 0) {
            calculate(a, b, op);
        } else {
            int result = calculate(a, b, op);
```

```
            System.out.println(result);
        }
    }
  }
}
```

Day 4 :

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

//Write your code here for Searchable interface:
interface Searchable {
  boolean searchByKeyword(String keyword);
}

//Write your code here for MediaItem abstract class:
abstract class MediaItem implements Searchable {
  private String title;
  private String releaseDate;
  private double rating;

  public MediaItem(String title, String releaseDate, double rating) {
    this.title = title;
    this.releaseDate = releaseDate;
    this.rating = rating;
```

```
    }

    public String getTitle() { return title; }
    public String getReleaseDate() { return releaseDate; }
    public double getRating() { return rating; }
}

//Write your code here for Movie class:
class Movie extends MediaItem {
    private String director;
    private List<String> cast;
    private int duration;

    public Movie(String title, String releaseDate, double rating,
            String director, List<String> cast, int duration) {
        super(title, releaseDate, rating);
        this.director = director;
        this.cast = cast;
        this.duration = duration;
    }

    public String getDirector() { return director; }
    public List<String> getCast() { return cast; }
    public int getDuration() { return duration; }

    @Override
    public boolean searchByKeyword(String keyword) {
        String kw = keyword.toLowerCase();
        if (getTitle().toLowerCase().contains(kw)) return true;
        if (director.toLowerCase().contains(kw)) return true;
        for (String member : cast) {
```

```
            if (member.trim().toLowerCase().contains(kw)) return true;
        }
        return false;
    }
}

public class Main {
    public static void sortByAttribute(List<Movie> movieList, String attribute, String order) {
        //Write your code here for sorting movie list
        Comparator<Movie> comparator;

        switch (attribute.toLowerCase()) {
            case "title":
                comparator = Comparator.comparing(m -> m.getTitle().toLowerCase());
                break;
            case "releasedate":
            case "release date":
            case "release_date":
                comparator = Comparator.comparing(Movie::getReleaseDate);
                break;
            case "director":
                comparator = Comparator.comparing(m -> m.getDirector().toLowerCase());
                break;
            case "duration":
                comparator = Comparator.comparingInt(Movie::getDuration);
                break;
            case "rating":
                comparator = Comparator.comparingDouble(Movie::getRating);
                break;
            default:
                // Handle unrecognized attribute gracefully - default to title sort
```

```
            System.err.println("Invalid attribute: " + attribute + ". Defaulting to title sort.");
            comparator = Comparator.comparing(m -> m.getTitle().toLowerCase());
            break;
        }

        if (order.equalsIgnoreCase("desc")) {
            comparator = comparator.reversed();
        } else if (!order.equalsIgnoreCase("asc")) {
            System.err.println("Invalid order: " + order + ". Defaulting to ascending.");
        }

        movieList.sort(comparator);
    }

//Non editable starts here
    public static void main(String[] args) {
        List<Movie> movieList = new ArrayList<>();

        try (Scanner scanner = new Scanner(System.in)) {
            int numberOfMovies = Integer.parseInt(scanner.nextLine());
            for (int i = 0; i < numberOfMovies; i++) {
                String title = scanner.nextLine();
                String releaseDate = scanner.nextLine();
                double rating = Double.parseDouble(scanner.nextLine());
                String director = scanner.nextLine();
                int duration = Integer.parseInt(scanner.nextLine());
                List<String> cast = new ArrayList<>(Arrays.asList(scanner.nextLine().
                split(",\\s*")));
                movieList.add(new Movie(title, releaseDate, rating, director, cast,
                duration));
            }
```

```
        String sortAttribute = scanner.nextLine();
        String sortOrder = scanner.nextLine();

        String searchKeyword = scanner.nextLine();
        movieList.stream()
            .filter(movie -> movie.searchByKeyword(searchKeyword))
            .forEach(movie -> System.out.println(movie.getTitle()));

        sortByAttribute(movieList, sortAttribute, sortOrder);
        movieList.forEach(movie -> System.out.println(movie.getTitle()));
    }
  }
}
//Non editable ends here
```

Day 5 :

```
enum ItemType {
  BEVERAGE,
  PASTRY
}

interface Orderable {
  double getPrice();
  String getName();
}

abstract class MenuItem implements Orderable {
  protected String name;
  protected double price;
```

```
    MenuItem(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    public String getName() {
        return name;
    }
}

class Beverage extends MenuItem {
    Beverage(String name, double price) {
        super(name, price);
    }
}

class Pastry extends MenuItem {
    Pastry(String name, double price) {
        super(name, price);
    }
}

class Order {
    private java.util.List<Orderable> items = new java.util.ArrayList<>();

    void addItem(Orderable item) {
        items.add(item);
```

```
        System.out.println("Order Added: " + item.getName());
    }

    double calculateTotal() {
        double total = 0;
        for (Orderable item : items) {
            total += item.getPrice();
        }
        System.out.println("Total Order Cost: " + total);
        return total;
    }
}
```

Day 6 :

```
import java.util.Scanner;

public class PasswordValidator {
    private static void validatePassword(String password) throws InvalidPasswordException {
        // write your logic here
        if (password.length() < 8) {
            throw new InvalidPasswordException("Password must be at least 8 characters long.");
        }

        boolean hasUpper = false;
        boolean hasLower = false;
        boolean hasDigit = false;
        boolean hasSpecial = false;

        for (char c : password.toCharArray()) {
            if (Character.isUpperCase(c)) hasUpper = true;
```

```
            else if (Character.isLowerCase(c)) hasLower = true;
            else if (Character.isDigit(c)) hasDigit = true;
            else if (!Character.isLetterOrDigit(c)) hasSpecial = true;  // Fix: explicitly check non-
alphanumeric
        }

        if (!hasUpper) {
            throw new InvalidPasswordException("Password must contain at least one uppercase letter.");
        }
        if (!hasLower) {
            throw new InvalidPasswordException("Password must contain at least one lowercase letter.");
        }
        if (!hasDigit) {
            throw new InvalidPasswordException("Password must contain at least one digit (0-9).");
        }
        if (!hasSpecial) {
            throw new InvalidPasswordException("Password must contain at least one special character
(@, #, $, %, etc.).");
        }
    }

// Non editable starts here
    public static void main(String[] args) {
        try (Scanner scanner = new Scanner(System.in)) {  // Fix: try-with-resources
            String password = scanner.nextLine();

            try {
                validatePassword(password);
                System.out.println("Password is valid.");
            } catch (InvalidPasswordException e) {
                System.out.println("Invalid Password: " + e.getMessage());
            }
```

```
        }
    }
}
// Non editable ends here

class InvalidPasswordException extends Exception {
    public InvalidPasswordException(String message) {
        super(message);
    }
}
```

Day 7 1st Question :

```
Thread t = new Thread(() -> {
    int sum = 0;

    for (int i = start; i <= end; i++) {
        if (i % 2 == 0) {
            sum += i;
        }
    }

    System.out.println(sum);
});

t.start();

try {
    t.join();
} catch (InterruptedException e) {
```

```
    e.printStackTrace();
}
```

Day 7 2nd Question :

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.Scanner;

public class PrintNumbersInSequence {
    // write your logic here
    private int n;
    private int current;
    private final Lock lock = new ReentrantLock();
    private final StringBuilder result = new StringBuilder();

    public PrintNumbersInSequence(int n) {
        // write your logic here
        this.n = n;
        this.current = 1;

        Runnable task = () -> {
            while (true) {
                lock.lock();
                try {
                    if (current > n) break;
                    if (result.length() > 0) result.append(" ");
                    result.append(current);
                    current++;
```

```
            } finally {
                lock.unlock();
            }
        }
    };

    Thread t1 = new Thread(task);
    Thread t2 = new Thread(task);
    Thread t3 = new Thread(task);

    t1.start();
    t2.start();
    t3.start();

    try {
        t1.join();
        t2.join();
        t3.join();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

    // Print only after all threads finish - thread-safe single println
    System.out.println(result.toString());
  }

// Non editable starts here
  public static void main(String[] args) throws InterruptedException {
    Scanner scanner = new Scanner(System.in);
```

```
        int n = scanner.nextInt();
        PrintNumbersInSequence obj = new PrintNumbersInSequence(n);
        scanner.close();
    }
}
// Non editable ends here
```

Day 8 :

```
import java.util.Scanner;

public class StudentIntersection {

    private static int numStudents(int A, int B, int C, int D, int E, int F, int G) {

        int numIntersection = G - A - B - C + D + E + F;

        return numIntersection;
    }

    // Non editable starts here
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int A = scanner.nextInt();
        int B = scanner.nextInt();
        int C = scanner.nextInt();
        int D = scanner.nextInt();
        int E = scanner.nextInt();
```

```
        int F = scanner.nextInt();
        int G = scanner.nextInt();

        int result = numStudents(A, B, C, D, E, F, G);
        System.out.println(result);

        scanner.close();
    }
}
```

Day 9 :

```
import java.util.*;

public class Main {

    public static int minLength(String S) {

        int[] freq = new int[256];

        for (char c : S.toCharArray()) {
            freq[c]++;
        }

        int max1 = 0, max2 = 0;

        for (int f : freq) {
            if (f > max1) {
                max2 = max1;
                max1 = f;
```

```
        } else if (f > max2) {
            max2 = f;
        }
    }

    return S.length() - max1 - max2;
  }

  // Non editable starts here
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    String S = scanner.nextLine();
    System.out.println(minLength(S));
    scanner.close();
  }
}
```

Day 17 4th Question :

```
SELECT customer_id, customer_name, email, phone_number
FROM (
  SELECT customer_id, customer_name, email, phone_number,
      ROW_NUMBER() OVER (PARTITION BY email ORDER BY customer_id) AS rn
  FROM CustomerContacts
) AS ranked_emails
WHERE rn = 1;
```

Day 17 5th Question :

```
SELECT d.doctor_id,
    d.doctor_name,
    COUNT(s.surgery_id) AS number_of_surgeries
FROM doctors d
LEFT JOIN surgeries s
ON d.doctor_id = s.doctor_id
GROUP BY d.doctor_id, d.doctor_name;
```